

Programming Project #2

EGRE246

Rational Numbers

1 Overview

Write a separately compiled C module to implement rational numbers as specified below. File `rat.h` (downloadable off the class web pages; you may not modify this file in any way!):

```
#include <stdbool.h>

#ifndef RAT_H
#define RAT_H

struct rtype {
    int n,d; // numerator, denominator
};

typedef struct rtype *rat;

rat createRat(int,int); // creates and returns a rat number

rat norm(const rat); // returns normalized number
rat reduce(const rat); // reduces number to lowest terms
int cmp(const rat r1, const rat r2); // returns -1 if r1 < r2,
                                     // 0 if r1 == r2,
                                     // 1 if r1 > r2

rat add(const rat r1, const rat r2); // r1 + r2
rat sub(const rat r1, const rat r2); // r1 - r2
rat mul(const rat r1, const rat r2); // r1 * r2
rat divide(const rat r1, const rat r2); // r1 / r2

rat inverse(const rat); // returns argument inverted

bool wellFormed(const rat); // returns true if number is legal, false otherwise

char *toString(const rat); // returns string representation of argument

#endif
```

Further specifications:

- Note that `rat` is defined using a `typedef` to be a pointer to a structure. Therefore if you define the following

```
rat r1;
```

`r1` will be a pointer to a 'struct `rType`' item.

- The function `createRat()` should create `rat` numbers at runtime using `malloc`.
- Legal rational numbers are to be stored exactly as they are created (i.e. they need not be stored reduced or in normal form).
- A `rat` number with a denominator of 0 is undefined. Functions `createRat` and `inverse` should check for denominators of 0 and if found should print out an appropriate error message and exit the program.
- Division by $\frac{0}{1}$ is undefined. Your `divide` function should check for this and if found should print out an appropriate error message and exit the program.
- In normal form denominators are always positive and the value 0 has no sign and is always normalized to 0/1 (i.e. 9/-3 normalized would be -9/3, 0/-3 would be 0/1).
- The `reduce` function reduces all values equal to 0 to 0/1.
- The arithmetic operations `add`, `sub`, `mul`, and `divide` always return a number reduced and in normal form.
- The function `cmp` compares two rational numbers `cmp(r1,r2)` returning -1 if `r1<r2`, 0 if `r1==r2`, and 1 if `r1>r2`.
- The function `toString` should return a `rat` number (with no judgements, i.e. it always outputs a string for rational number be it well-formed or not) represented in the traditional way with a '/' between the numerator and denominator, e.g. $-\frac{2}{3}$ would be represented by the string "-2/3".
- All operations (i.e. `norm`, `reduce`, `cmp`, `add`, `sub`, `mul`, `divide`, and `inverse`) should only be attempted on well-formed `rat` numbers. If an illegal `rat` number is encountered through testing with `wellFormed` in these routines you should print an appropriate error message and exit the program.
- You should exit your program using `exit(EXIT_FAILURE)` and all error messages should contain the word "error" (any case) somewhere in them.

2 Sample Driver

Here is a sample driver program `ratDriver.c` that can be used as the first step in testing your program:

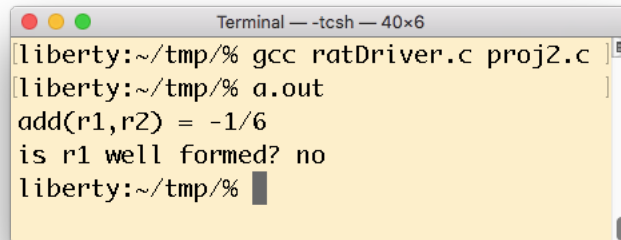
```
#include <stdio.h>
#include "rat.h"

int main(void) {
```

```

    rat r1 = createRat(2,3), r2 = createRat(5,-6);
    printf("add(r1,r2) = %s\n",toString(add(r1,r2)));
    r1->d = 0;
    printf("is r1 well formed? %s\n",wellFormed(r1)?"yes":"no");
}

```



```

Terminal — -tcsh — 40x6
liberty:~/tmp/% gcc ratDriver.c proj2.c
liberty:~/tmp/% a.out
add(r1,r2) = -1/6
is r1 well formed? no
liberty:~/tmp/%

```

3 Deliverables

You should only turn in your rational module implementation file (i.e. the file containing the implementation of everything defined in `rat.h`)! This means that the code you submit will not contain a `main` function. I will test your code with my own driver test program. Submit your project via Canvas. Note you need not turn in an executable file, your driver program, or file `rat.h`! You may work in pairs on this project; if you work with a partner please only submit one project under both of your names (with both of your names in a comment at the beginning of the submitted file).